

De zéro à l'API "REST"

- progression guidée en moins de 2h
- 20 participants

Mots Clés

MicroPython · Python embarqué · Thonny
RaspberryPi PicoW · MakerPi PicoW (SHT30) | PicoBricks (SHTC3)
Asyncio · WiFi · Détection d'anomalie embarquée

Eric DUVIEILBOURG · IR CNRS / LEMAR



↓ github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/supports/

Atelier_MicroPython_PicoW_beta

Construire pas à pas

capteur I2C → asyncio → WiFi "AP" → API "REST" → "IA" détection d'anomalie ou statistique locale



Capteur I2C

SHT30 (MakerPi Pico)
SHTC3 (PicoBricks)



Pico W

Python · asyncio · WiFi · IA

I2C

async

HTTP

z-score

microcontrôleur
détection anomalie
tout local · AP WiFi



Votre téléphone



Votre PC

<http://192.168.4.1>
dashboard live

Aucune infrastructure · Aucun Cloud · Aucun abonnement

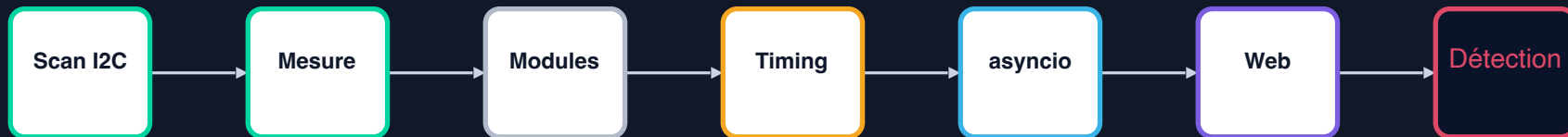
Pourquoi MicroPython ?

Fonctionnalité	Python (PC)	MicroPython (Pico)
Syntaxe Python	✓	✓ identique
f-strings, list comp	✓	✓ depuis v1.22
asyncio / await	asyncio	uasyncio — même API
numpy / scipy	✓	ulab (sous-ensemble, compilé en C)
RAM disponible	Go	264 Ko — gc.collect() !
Accès hardware (I2C, SPI...)	via libs tierces	machine.Pin, I2C, SPI...
Filesystem	open(), os	open(), os — identiques

Règle : si ça marche en *Python*, ça marche probablement en *MicroPython* (sauf RAM et stdlib)

Déroulé de l'atelier

chaque partie se termine par un fichier précis à lancer dans **Thonny**



Progression

1. Vérifier le **bus I2C** et l'adresse du **capteur**
2. Lire plusieurs **mesures** et contrôler les valeurs obtenues
3. Structurer le code par modules,
maîtriser le **timing**,
puis ajouter **asyncio**, **web** et **détection locale**

/JDEV2026

```
steps/01...07  
main.py (final)  
slide/  
LICENSE*  
README.md
```

Code à tester en fin de chaque partie

ouvrir et lancer le **main.py** indiqué dans **/JDEV2026/steps/xx_*/main.py**

Accès au code, GitHub et licence

Le dépôt (bêta version*) sert à distribuer les versions étape par étape et l'application finale

ORGANISATION

- Dépôt GitHub JDEV2026-MicroPython-PicoW
- Tags* par étape : step-01, step-02... puis final
- Accès en lecture pendant l'atelier*

```
git tag step-01-i2c-scan
git tag step-02-sht30-5-mesures
git tag step-02-shtc-5-mesures
git tag step-03-code-structure
git tag step-04-timing-ticks
git tag step-05-asyncio
git tag step-06-web-api
git tag step-07-ia-zscore
git tag final
```

LICENCE

- Supports : CC BY-NC-ND 4.0
- Code : licence pédagogique JDEV2026 source-available
- Attribution obligatoire et redistribution modifiée uniquement avec autorisation

© 2026 Eric Duvieilbourg — CNRS / LEMAR

Usage pédagogique JDEV2026 autorisé

Attribution obligatoire

Redistribution de versions modifiées interdite sans autorisation écrite

Carte | Arborescence des fichiers dans "Thonny"

Double-clic → éditeur · Clic droit → Upload to /

```

Pico W – arborescence dans Thonny

/ (racine du Pico – fichiers kit intacts)
main.py          ← kit Pico Bricks · NE PAS TOUCHER
picobricks.py    ← kit Pico Bricks · NE PAS TOUCHER

JDEV2026/
  main.py        ← point d'entrée
  config.py      ← USER_INITIALS = 'ed'
  acquisition.py ← SHT30 (Maker) / SHTC3 (PicoBricks)
  web_ap.py      ← serveur HTTP & WiFi AP
  ai/detector.py ← z-score embarqué
  lib/sht30.py   ← driver SHT30 (Maker)
  lib/picobricks_sensors.py ← driver SHTC3 (PicoBricks)
  static/index.html ← dashboard navigateur
  steps/01...07...py ← scripts par étapes
  
```

 Point d'entrée

 Capteur

 Web/WiFi

 IA

 Config

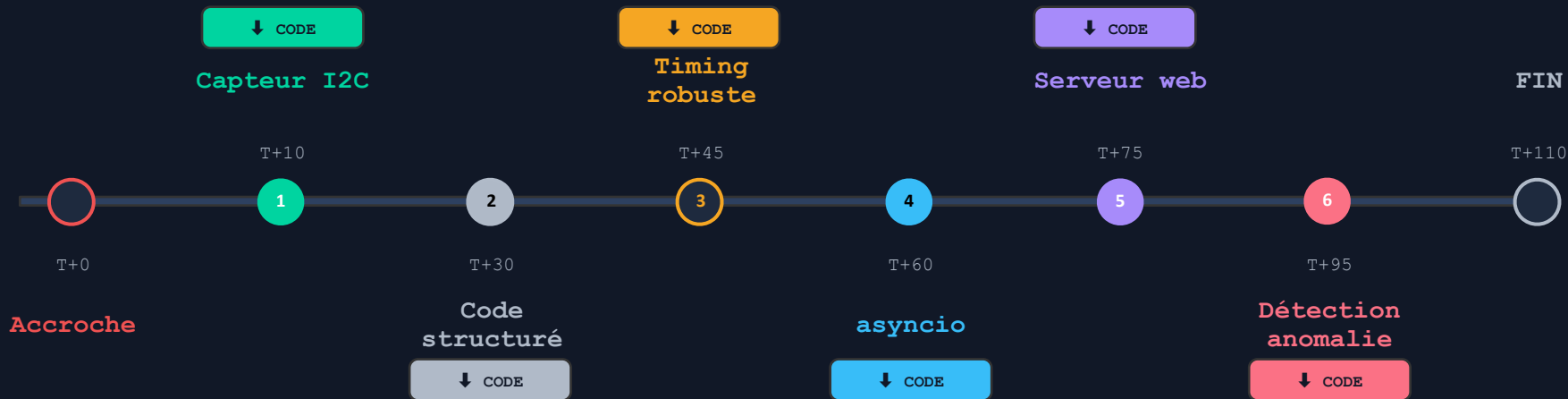
Thonny

Votre PC

 **Pico W**

-  **JDEV2026/**
-  main.py
-  config.py
-  acquisition.py
-  web_ap.py
-  ai/
-  lib/
-  static/

Du capteur au dashboard "IA"



Chaque étape clé : *code projeté + branche GitHub à récupérer + exercice guidé*

Point de départ

vérifier que le Pico, **Thonny** et le capteur (soit le ENVIII, soit le TEMP&HUM) sont opérationnels



Checklist minute : 0.5

1. Le dossier **/JDEV2026** est copié sur le Pico
2. **Thonny** voit le port **MicroPython**
3. Le **capteur** est branché en **Grove I2C**, si rien ne répond :
vérifier le connecteur **Grove** sur **GP0/GP1 (MarkerPico)**
sur **GP4/GP5 (PicoBricks)**

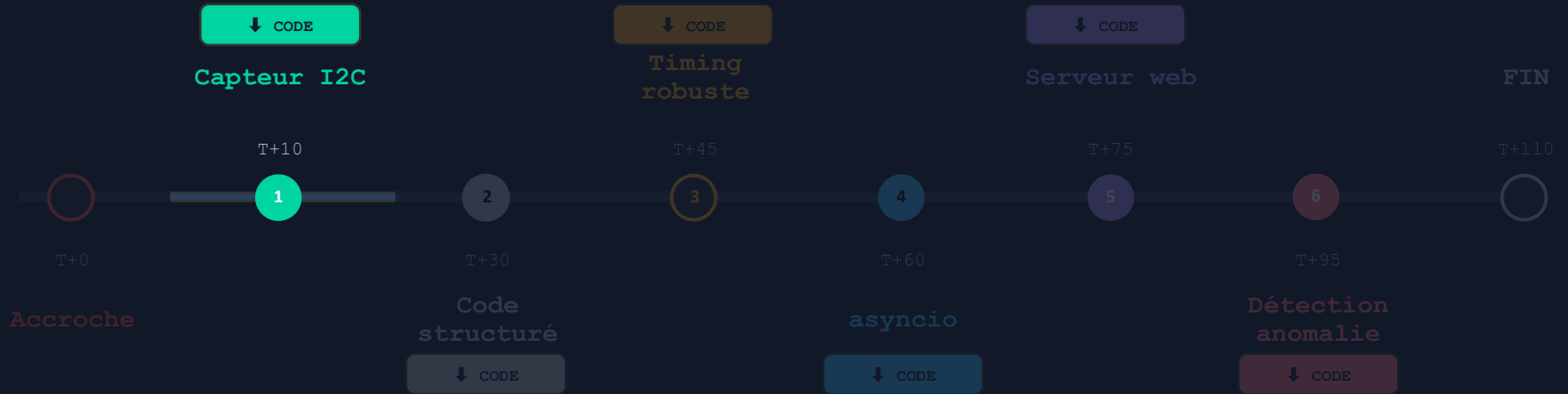
Raccourcis Thonny utiles

```
Ctrl+S : sauvegarder sur Pico  
F5      : lancer le script  
Ctrl+C  : stopper le script  
Ctrl+D  : soft reboot
```

↓ github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026/

[steps/01_i2c_scan/main.py](#)

Du capteur au dashboard "IA"



Etape 1 : Capteur I2C

Lire un capteur "I2C" : SHT30 . SHTC3

Protocole I2C

SDA données (bidirectionnel)

SCL horloge (maître→esclave)

VCC alimentation 3.3V

GND masse



2 fils utiles : SDA + SCL



Adresse 0x70 (SHTC3) · 0x44 (SHT30)



machine.I2C — même logique que smbus2 (PC)



i2c.scan() → preuve que le capteur répond

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/02_sht30_5_mesures/main.py](#)

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/02_shtc3_5_mesures/main.py](#)

Correction principe : 5 mesures, min/max

Appeler `sensor.measure()` 5 fois · stocker les températures · afficher min et max

acquisition.py

```
# Dans le REPL Thonny (console >>>)
from machine import I2C, Pin

# Pico Bricks : from JDEV2026.lib.picobricks_sensors import SHTC3

i2c = I2C(0, scl=Pin(5), sda=Pin(4))

# Maker Pi Pico : from JDEV2026.lib.sht30 import SHT30

temps = []
for i in range(5):
    temp, hum = sensor.measure()
    temps.append(temp)
    print(f" {i+1}/5 → {temp:.2f}°C {hum:.1f}% »)

print(f"Min : {min(temps):.2f} °C")
print(f"Max : {max(temps):.2f} °C")
```

Sortie REPL

```
# Résultat attendu :

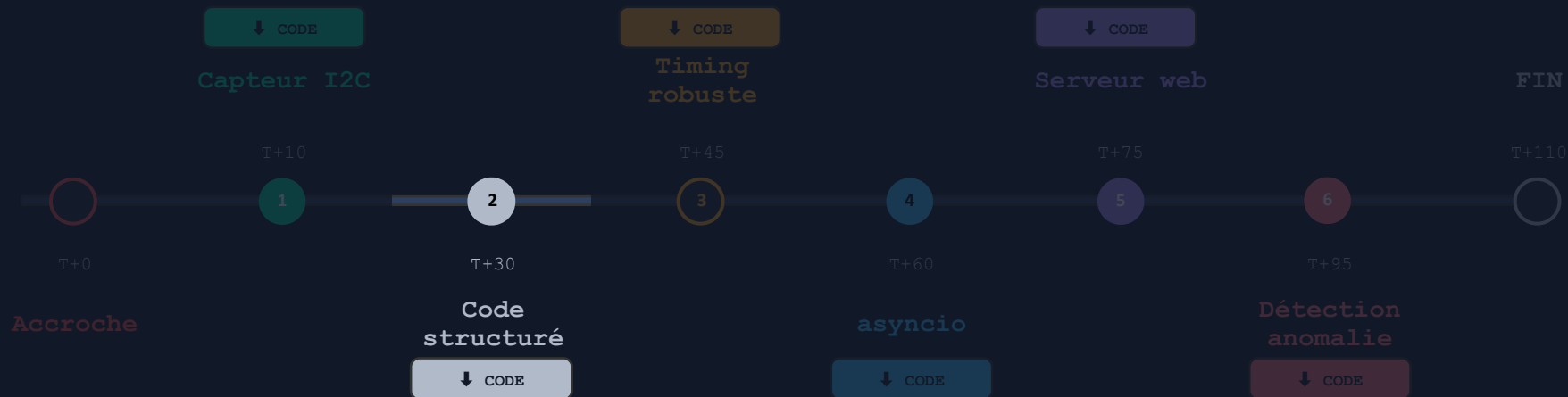
1/5 → 22.34°C 58.1%
2/5 → 22.36°C 57.9%
3/5 → 22.35°C 58.2%
4/5 → 22.33°C 58.0%
5/5 → 22.37°C 58.1%

Min : 22.33°C
Max : 22.37°C
```

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/02_sht30_shtc3_5_mesures/main.py](#)

Du capteur au dashboard "IA"



Etape 2 : Code Structuré et modulaire

Modules MicroPython : mêmes réflexes qu'en Python-C



Mauvaise habitude embarquée

● ● ● main.py — flat

```
from machine import I2C, Pin
from JDEV2026.lib.picobricks_sensors import SHTC3

i2c = I2C(0, scl=Pin(5), sda=Pin(4))
s = SHTC3(i2c)

while True:
    print(s.temperature())
    time.sleep(2)
```



Python propre — fonctionne en MicroPython

● ● ● acquisition.py — module

```
# acquisition.py
from machine import I2C, Pin
from JDEV2026.lib.picobricks_sensors import SHTC3

def init_sensor():
    i2c = I2C(0, scl=Pin(5), sda=Pin(4))
    return SHTC3(i2c)

def read_sensor(sensor):
    return sensor.temperature(), sensor.humidity()

if __name__ == "__main__":
    sen = init_sensor()
    print(read_sensor(sen))
```

Vous reconnaissez ce code ! C'est intentionnel : vos réflexes Python fonctionnent ici

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/03_code_structure/main.py](#)

Modules MicroPython

séparer initialisation matérielle, lecture capteur et script principal



- . Le **main.py** orchestre
- . Le **module_acquisition.py** isole le matériel
- . La version finale gardera ce principe de séparation

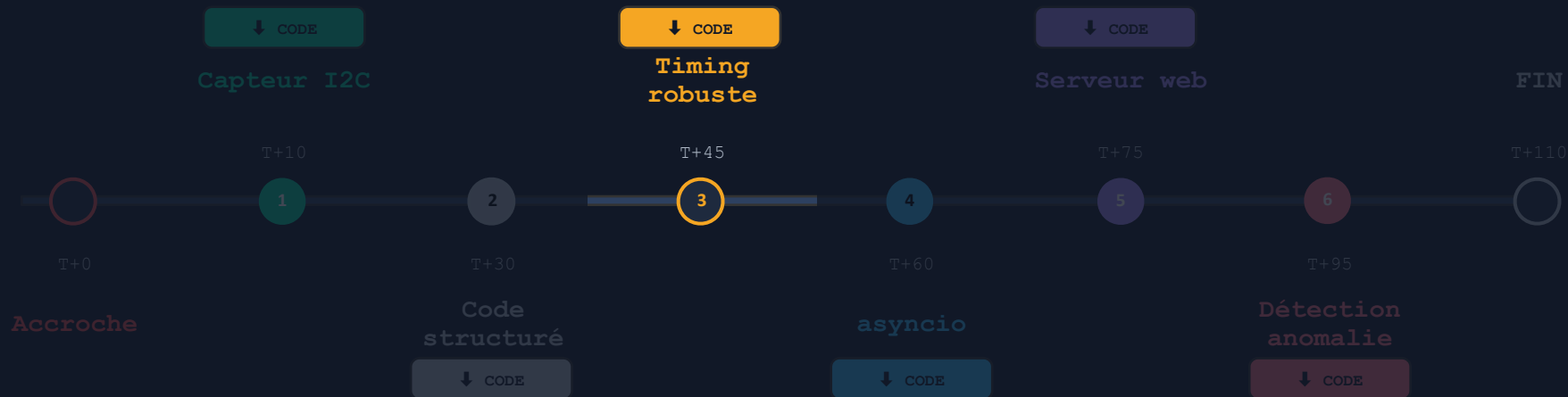
```
# acquisition.py

def init_sensor():
    i2c = I2C(0, sda=Pin(0), scl=Pin(1))
    return SHT30(i2c, address=0x44)

def read_sensor(sensor):
    return sensor.measure()
```

Bonnes pratiques !!!

Du capteur au dashboard "IA"



Etape 3 : Timing robuste

Timing précis : ticks_ms() vs sleep()

conserver une période d'échantillonnage stable

❌ time.sleep(5) dérive

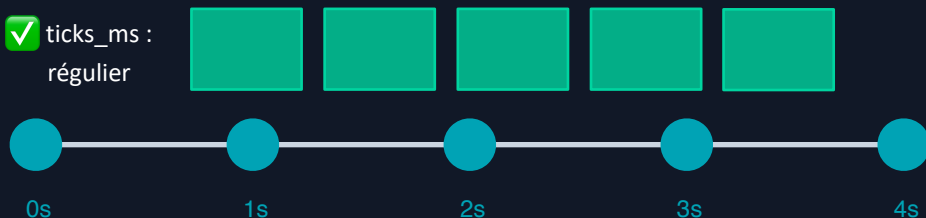
sleep(5) attend 5s fixes
 Mais read_sensor() prend ~15ms
 → dérive 15ms × N mesures / heure
 → série temporelle inutilisable après quelques heures

❌ sleep :
dérive



✅ ticks_ms() — temps absolu

✅ ticks_ms :
régulier



04_timing_ticks/main.py

```
import utime

PERIOD_MS = 2000

t_next = utime.ticks_ms()

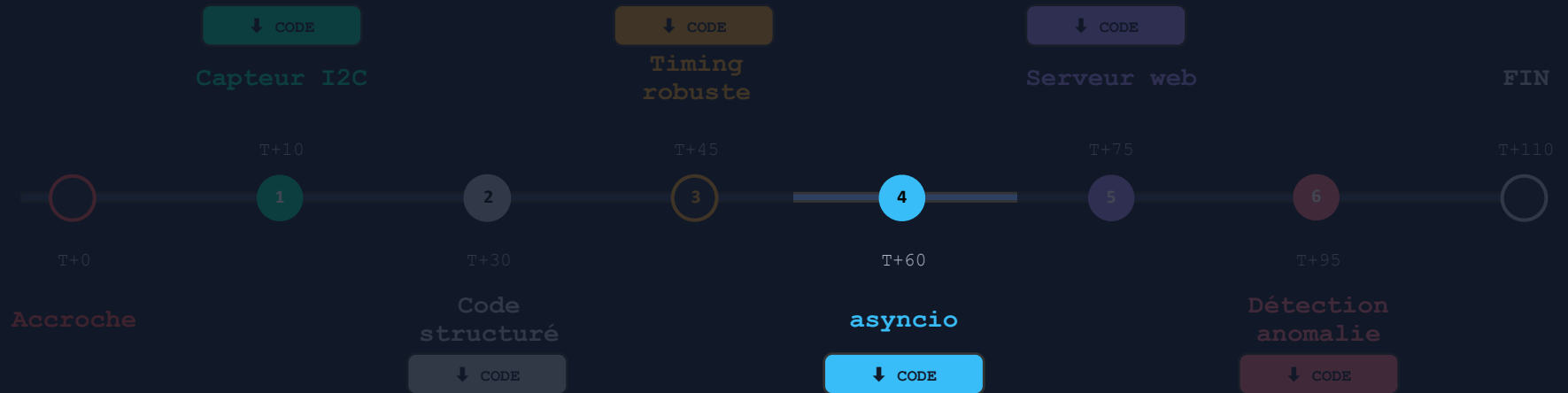
while True:
    t, h = read_sensor(sensor)
    log(t, h)

    # Timing précis sans dérive
    t_next = utime.ticks_add(t_next, PERIOD_MS)
    delay = utime.ticks_diff(t_next, utime.ticks_ms())
    if delay > 0:
        utime.sleep_ms(delay)
```

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

steps/04_timing_ticks/main.py

Du capteur au dashboard "IA"



Etape 4 : asyncio ou exécuter plusieurs tâches coopératives

Multitâche coopératif : "uasyncio"

`asyncio.gather()``blink(200ms)``log_sensor(2s)``web_server()``task_ai()``CPU libre (pour await)`

Même API que asyncio Python 3.4+

```
main.py — asyncio

import uasyncio as asyncio

async def blink(period_ms):
    while True:
        led.toggle()
        await asyncio.sleep_ms(period_ms)

async def log_sensor(period_s):
    while True:
        t, h = read_sensor(sensor)
        log_measurement(t, h)
        await asyncio.sleep(period_s)

async def main():
    await asyncio.gather(
        blink(200), log_sensor(2),
        web_server(), task_ai())

asyncio.run(main())
```

Modifier la fréquence "LED"

asyncio est coopératif : deux tâches indépendantes, un seul cœur CPU

1

Ouvrez JDEV2026/steps/05_asyncio_led_capteur/main.py

Double-clic dans le panneau Pico · onglet s'ouvre dans l'éditeur

2

Trouvez la ligne `blink(200)`

Ctrl+F → cherchez 'blink' — elle est dans `asyncio.gather()`

3

Changez 200 en 50

`blink(200)` → `blink(50)`

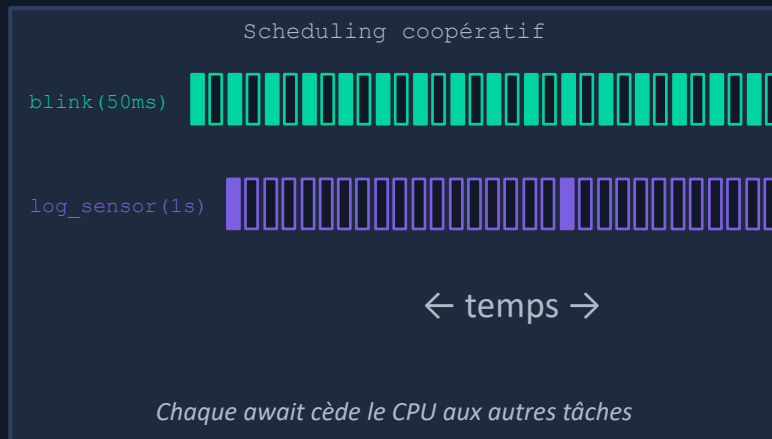
4

Sauvegardez et relancez

Ctrl+S puis F5



La mesure capteur continue à 1Hz même quand la LED clignote à 20Hz, que constatez-vous ?



github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/05_asyncio_led_capteur/main.py](#)

Modifier la fréquence "LED"

asyncio est coopératif : deux tâches indépendantes, un seul cœur CPU

1

Ouvrez JDEV2026/steps/05_asyncio_led_capteur/main.py

Double-clic dans le panneau Pico · onglet s'ouvre dans l'éditeur

2

Trouvez la ligne `blink(200)`

Ctrl+F → cherchez 'blink' — elle est dans `asyncio.gather()`

3

Changez 200 en 50

`blink(200)` → `blink(50)`

4

Sauvegardez et relancez

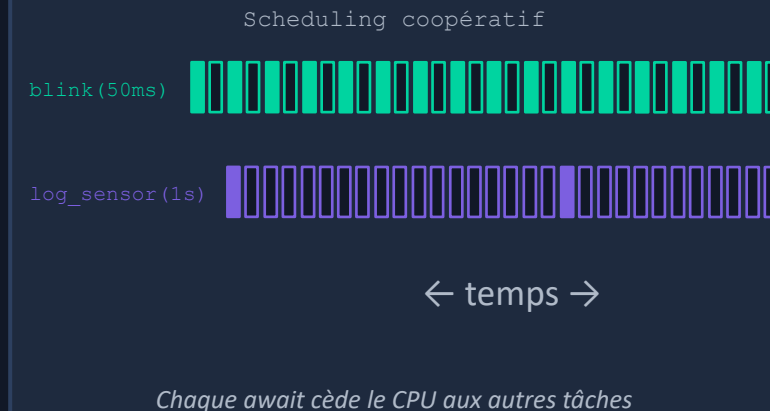
Ctrl+S puis F5



La mesure continue à 1Hz même quand la LED clignote à 20Hz.



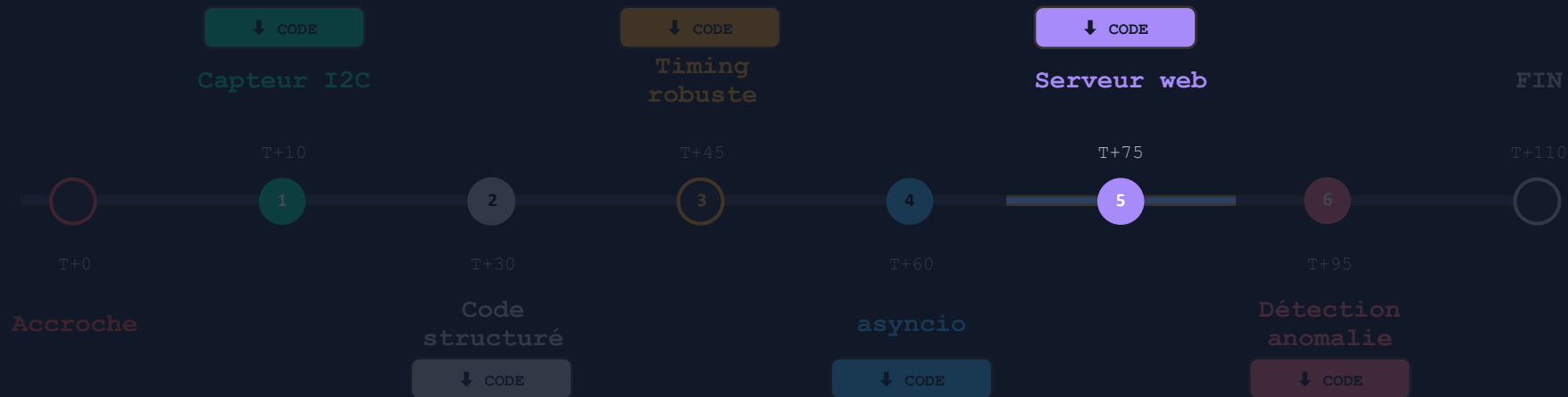
Le résultat est perturbée, POURQUOI ? QUELLES SOLUTIONS ?



github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/05_asyncio_all/main_async.py](#)

Du capteur au dashboard "IA"



Etape 5 : Serveur Web HTTP

API "REST" sur Pico W : WiFi mode AP



Votre smartphone

192.168.4.2

≈ ≈ WiFi ≈ ≈

edPicoW-JDEV
pass: micropython

Pico W — AP mode

192.168.4.1

GET /data

JSON mesure + état IA

GET /alerts

Historique anomalies

GET /config

Paramètres actifs

GET /

Dashboard HTML

web_ap.py

```
import network, socket

ap = network.WLAN(network.AP_IF)
ap.active(True)
ap.config(essid='edPicoW-JDEV',
          password='micropython')

async def web_server():
    s = socket.socket()
    s.bind(('', 80))
    s.listen(1)
    while True:
        cl, _ = s.accept()
        req = cl.recv(512)
        if b'/data' in req:
            t = sensor.temperature()
            cl.send(f'{t:.2f}')
        cl.close()
```

Sans Cloud · Sans infrastructure · Sans Internet · 8€ de matériel

Exercice : Votre premier route JSON

Ajoutez la route `/temp` : comprendre le dispatch HTTP embarqué

1

Ouvrez `JDEV2026/05_web_api/web_server.py`

Double-clic sur `web_server.py` dans le panneau Pico

2

Trouvez le bloc `if b'/data' in req`

C'est le dispatcher de routes — 3 lignes

3

Testez ce bloc dans l'url



C'est une vraie API REST — URL, verbe HTTP, réponse JSON, sans état

Steps/05_web_api/web_server.py

```
# après le bloc /data existant :  
elif b'/temp' in req:  
    t = sensor.temperature()  
    cl.send(  
        f'{"temp":{t:.1f}},'  
        f'"unit":"C",'  
        f'"ts":{utime.time()}}'  
    )
```



Testez : `http://192.168.4.1`



Testez : `http://192.168.4.1/data`



Testez : `http://192.168.4.1/temp`

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

steps/06_web_api/main.py

Testez les différents route JSON

 <http://192.168.4.1>



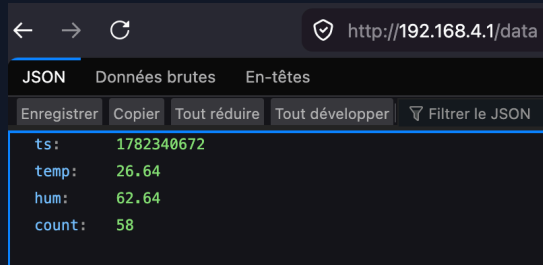
Pico W JDEV2026

24.9 °C

Humidité : 66 %

Mesures : 15

 Testez : <http://192.168.4.1/data>

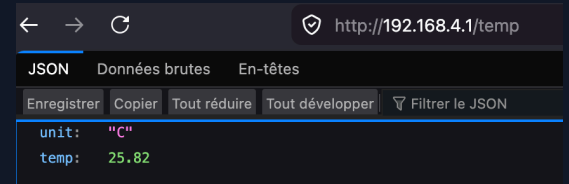


JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer Filtre le JSON

```
ts: 1782340672
temp: 26.64
hum: 62.64
count: 58
```

 Testez : <http://192.168.4.1/temp>



JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer Filtre le JSON

```
unit: "C"
temp: 25.82
```

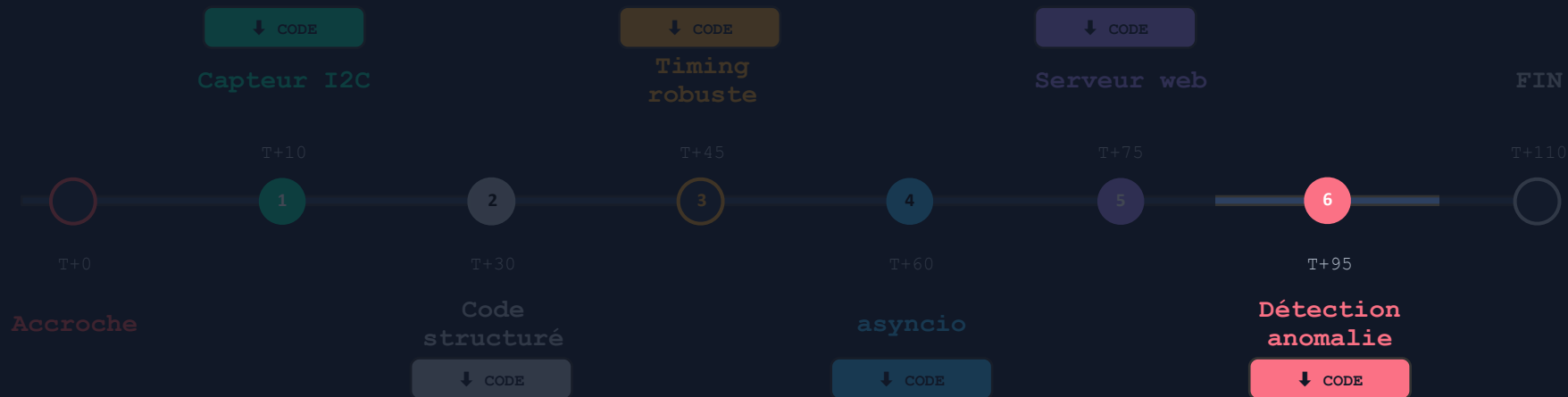
github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/06_web_api/main.py](#)

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/06_web_api/main_async.py](#)

Du capteur au dashboard "IA"



Etape 5 : Serveur Web HTTP

Détection d'anomalie : ce que c'est vraiment

⚠ Rigueur : ce qu'on implémente n'est PAS du Machine Learning, c'est de la statistique descriptive temps réel



Ce qu'on fait ici

Statistique descriptive
(z-score, fenêtre glissante)

Règle basée sur un seuil
statistique → rule-based
anomalydetection

✓ Dans l'atelier



IA légère / TinyML

Inférence de modèle
compilé sur μC

Modèle entraîné offline
(Edge Impulse, ONNX)
inférence < 10ms

→ Étape suivante



Machine Learning

Entraînement +
inférence sur PC/GPU

numpy, scikit-learn,
PyTorch — requiert
puissance de calcul

✗ Pas sur un μC

Le CSV généré pendant cet atelier peut servir de dataset pour Edge Impulse (TinyML) — étape suivante

z-score glissant : la formule et le code

$$z = | \text{valeur} - \mu_{\text{fen\^etre}} | / \sigma_{\text{fen\^etre}}$$

fen\^etre = 20 mesures
seuil = 2.5σ

15

lignes

de code Python pur

2

Ko

RAM utilisée

<5

ms

par mesure

*Terminologie correcte : d\^etection d'anomalie par seuillage statistique
(statistical anomaly detection, rule-based monitoring)*

```
ai/detector.py

# ai/detector.py
import math

class AnomalyDetector:
    def __init__(self, window=20, thr=2.5):
        self.buf = []
        self.n = window
        self.thr = thr

    def push(self, v):
        self.buf.append(v)
        if len(self.buf) > self.n:
            self.buf.pop(0)
        m = sum(self.buf) / len(self.buf)
        s = math.sqrt(sum((x-m)**2
            for x in self.buf) / len(self.buf))
        return s > 0 and abs(v-m) > self.thr*s
```

Exercice Déclencher une anomalie physique

Provoquer un écart de température et observer la détection en temps réel sur la console Thonny

1

Attendez 20 mesures

Pour établir sa baseline

2

Mettez la main sur le capteur

Ou soufflez 3 secondes

3

Observez la console Thonny

Le message anomalie doit apparaître

Console

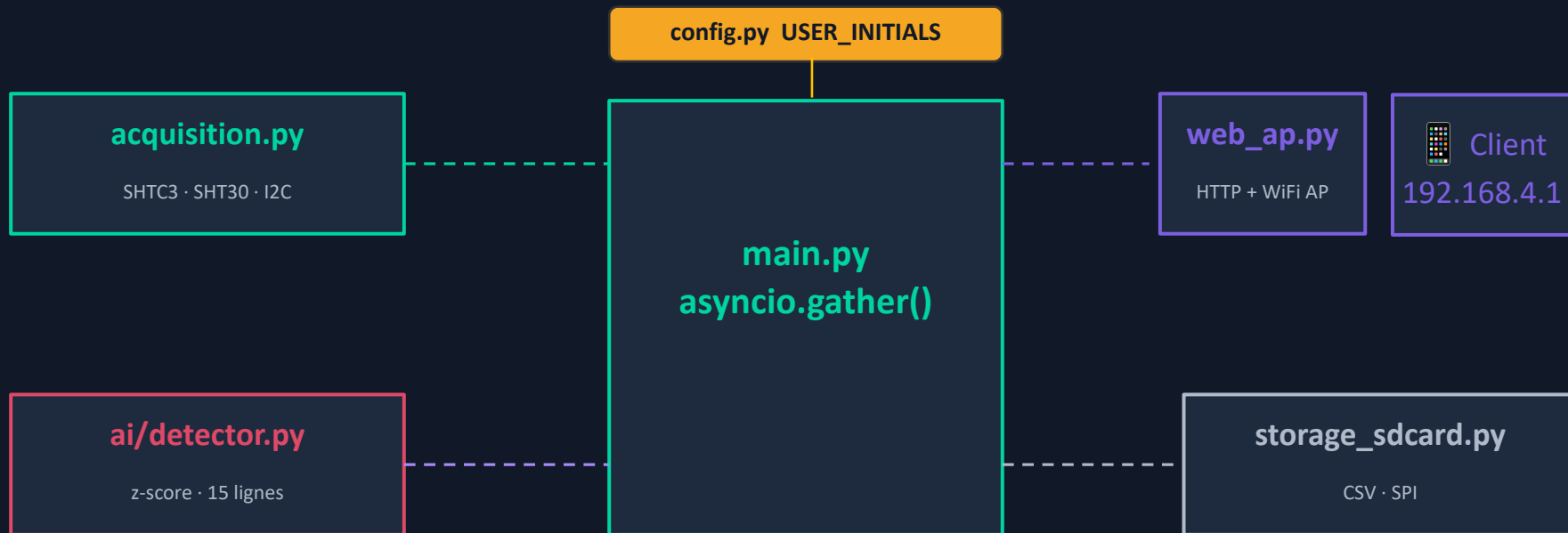
```
MPY: soft reboot
Mémoire libre au démarrage : 192064 octets
Périphériques I2C détectés : ['0x22', '0x3c', '0x70']
✓ Capteur utilisé : SHTC3 / PicoBricks

=== Étape 7 – détection d'anomalie z-score ===
Attendez la baseline, puis soufflez sur le capteur.
Ce n'est pas un modèle prédictif : c'est un détecteur statistique embarqué.
001 | T=24.50 °C | mean=24.50 | sigma=0.000 | z=0.00
002 | T=24.53 °C | mean=24.51 | sigma=0.015 | z=0.00
003 | T=24.48 °C | mean=24.50 | sigma=0.021 | z=0.00
004 | T=24.51 °C | mean=24.50 | sigma=0.018 | z=0.00
005 | T=24.52 °C | mean=24.51 | sigma=0.017 | z=0.00
006 | T=24.52 °C | mean=24.51 | sigma=0.016 | z=0.00
007 | T=24.47 °C | mean=24.50 | sigma=0.021 | z=0.00
008 | T=24.53 °C | mean=24.51 | sigma=0.021 | z=1.07
009 | T=24.50 °C | mean=24.51 | sigma=0.020 | z=0.33
010 | T=27.04 °C | mean=24.76 | sigma=0.760 | z=3.00 Δ ANOMALIE
011 | T=28.20 °C | mean=25.07 | sigma=1.226 | z=2.55 Δ ANOMALIE
012 | T=28.86 °C | mean=25.39 | sigma=1.573 | z=2.21
013 | T=28.87 °C | mean=25.66 | sigma=1.773 | z=1.81
014 | T=27.76 °C | mean=25.81 | sigma=1.793 | z=1.09
```

github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

[steps/07_la_zscore/main.py](#)

Architecture du projet JDEV2026 complet



github.com/eduvieilbourg/JDEV2026-MicroPython-PicoW/JDEV2026

main.py

Architecture du projet JDEV2026 complet

Provoquer un écart de température et observer la détection en temps réel sur votre téléphone

1

Attendez 20 mesures

Le détecteur a besoin de 20 mesures pour établir sa baseline

2

Soufflez sur le capteur / Mettez la main

Approchez la bouche à 5cm du capteur, soufflez 3 secondes

3

Observez le REPL (Thonny)

Le message anomalie doit apparaître

4

Ajustez le seuil dans config.py

AI_ZSCORE_THRESH = 2.0 → plus sensible · 3.0 → moins sensible

Ce qui se passe dans le code

`sensor.temperature()` $T = 28.4^{\circ}\text{C}$

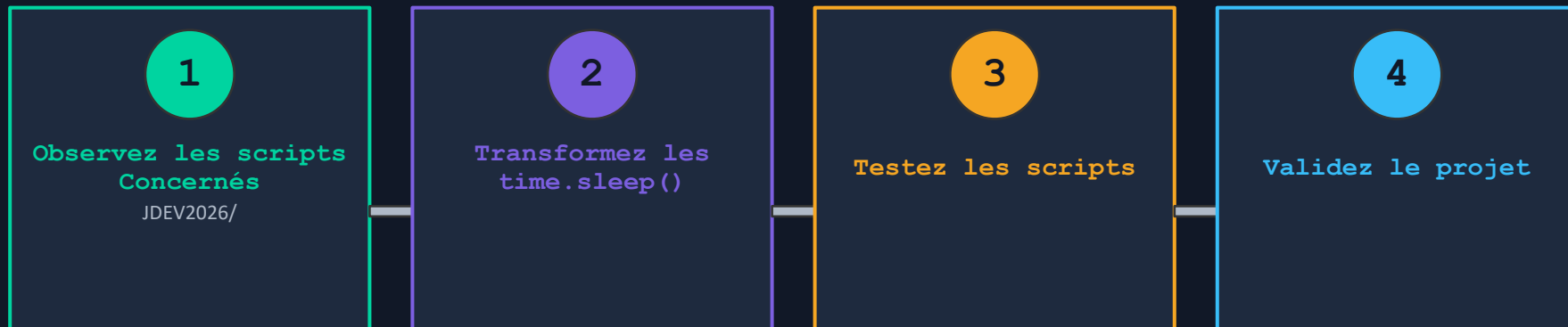
`detector.push(28.4)` $z = 3.7 > 2.5\sigma$ ✓

`alerts.fire()` cooldown OK → alerte

`shared_data['anomaly']` True

`/data → navigateur` anomaly:true → 

Utilisez le full asyncio sur l'ensemble des scripts



© **Licence CC BY-ND 4.0**

Creative Commons Attribution — NoDerivatives 4.0 International

- ✓ Redistribution autorisée avec attribution
- ✓ Usage commercial autorisé
- ✗ Modification interdite sans permission

Eric DUVIEILBOURG
CNRS / LEMAR
jdev2026

Ce que vous savez maintenant faire



Capteur I2C

`machine.I2C` · SHTC3 / SHT30



Modules Python

`import` · `def` · `__main__`



Timer sans dérive

`ticks_ms` · `ticks_diff`



Multitâche `asyncio`

`gather` · `await` · coopératif



Serveur HTTP REST

`AP_IF` · `socket` · JSON



Monitoring statistique

`z-score` · `rule-based` · 15 lignes

Sur un microcontrôleur à 8€ · Sans OS · Avec vos réflexes Python existants

Et ensuite — la formation complète



MQTT / LoRa / BLE

Protocoles IoT
industriels



TinyML

Edge Impulse
modèles < 100Ko



Sécurité

TLS · auth
apikey



Edge → Cloud

InfluxDB
Grafana



Tests embarqués

pytest · mocking
hardware



Datalogger long terme

CSV · SD · rotation
compression

Merci !

Le code complet est sur GitHub



Eric DUVIEILBOURG · CNRS / LEMAR · eric.duvieilbourg@cnrs.fr